

Diário de Bordo

Trabalho Prático I · Estruturas de Dados I (DS130)

Sistema de C

Semana 1

Responsabilidade

Trabalho individual. Todas as etapas foram feitas por mim: arquitetura, implementação dos três módulos, testes e documentação.

O que foi feito

- Leitura do enunciado e levantamento das funcionalidades obrigatórias

- Decisão d

A decisão de projeto

Precisei escolher duas vezes. Primeiro entre lista estática e dinâmica para os participantes, depois entre lista encadeada comum e lista com nó cabeçalho.

Descartei a estática logo de cara porque eu não tenho como saber quantas pessoas vão se inscrever num evento. Eu teria que chutar um MAX e correr o risco de estourar ou de desperdiçar um monte de espaço reservado à toa.

Entre a encadeada comum e a com cabeçalho, escolhi a com cabeçalho porque as duas coisas que o sistema mais faz são inscrever e remover participante. Na lista comum, remover o primeiro elemento precisa de um if separado, porque ele mexe no próprio ponteiro de início da lista. Com o sentinela sempre existe um nó anterior, mesmo quando o que você quer remover é o primeiro. Aí o código fica um só.

Para a lista de eventos eu usei encadeada simples de propósito, para comparar as duas estratégias. Sobre eventos o sistema só cadastra e busca por código, não remove com frequência e não precisa voltar, então uma lista dupla seria ponteiro a mais sem ganho nenhum.

Dificuldade encontrada

A maior dificuldade dessa semana foi ponteiro, principalmente a seta. Eu travei bastante em entender que um ponteiro não guarda o valor, ele guarda o endereço de onde o valor está. E que a seta significa "vai até esse endereço e pega o campo de lá". Enquanto eu li anterior->proximo como se fosse uma coisa só, nada fez sentido.

A segunda dificuldade veio logo em seguida, quando entrei na lista com cabeçalho. Eu confundia o cabecalho com o primeiro elemento da lista. Demorei pra firmar que o ponteiro da struct aponta sempre pro sentinela, e que o primeiro participante de verdade é o cabecalho->proximo.

Como resolvi

O que mais me ajudou foi acompanhar uma operação passo a passo inventando endereços de memória e anotando o que cada ponteiro guardava em cada linha. Botar número no lugar da seta tirou a mágica da coisa. Quando eu vi que l->inicio era só o número 200, e que l->inicio->dado era "vai no 200 e pega o dado", a seta parou de me assustar.

Depois disso eu voltei no guia da lista com cabeçalho que o professor disponibilizou e reli com calma. A regra que resolveu a minha confusão entre cabeçalho e primeiro real foi essa: se a função lê dados, o percurso começa em cabecalho->proximo; se ela precisa achar um nó anterior pra religar ponteiro, começa no cabecalho. Usei essa regra pra decidir onde começar cada laço do trabalho inteiro.

Trecho de código explicado nas minhas palavras

```
int lp_remove_ra(ListaParticipantes *l, const char *ra, Participante *saida)
{
    NoP *anterior = l->cabecalho;
    NoP *removido;

    while (anterior->proximo != NULL &&
           strcmp(anterior->proximo->dados.ra, ra) != 0) {
        anterior = anterior->proximo;
    }

    if (anterior->proximo == NULL) return 0;

    removido = anterior->proximo;
    if (saida != NULL) *saida = removido->dados;

    anterior->proximo = removido->proximo;
    free(removido);

    l->tamanho--;
    return 1;
}
```

Essa é a função que mostra melhor por que eu escolhi a lista com cabeçalho, então escolhi ela pra explicar.

O ponteiro anterior começa no sentinela. Isso é o ponto principal. Como o sentinela sempre existe, remover o primeiro inscrito da lista usa exatamente o mesmo código que remover qualquer outro. Numa lista sem cabeçalho eu precisaria de um if só pra esse caso, porque o primeiro elemento é o único que não tem ninguém antes dele.

O while não para no nó que tem o RA procurado. Ele para no nó anterior a esse. Isso confunde quando você olha rápido, mas faz sentido: quem precisa mudar de ponteiro é o nó de trás, porque é ele que vai ter que pular o removido. Por isso a condição olha anterior->proximo->dados.ra e não anterior->dados.ra.

Se o laço chegar até o fim e anterior->proximo for NULL, significa que percorri tudo e aquele RA não estava inscrito. Aí a função devolve 0 e quem chamou mostra a mensagem de erro.

A linha anterior->proximo = removido->proximo é a remoção propriamente dita. O nó de trás passa a apontar direto pro nó da frente, pulando o que vai sair. Nesse momento o nó removido já está fora da corrente, só que a memória dele ainda está alocada.

Por último vem o free. E ele vem por último de propósito. Se eu chamasse free(removido) antes da linha de cima, a linha de cima tentaria ler removido->proximo de um bloco de memória que já foi devolvido pro sistema. Isso é uso de memória já liberada, e o pior é que às vezes funciona em um teste e quebra em outro. Então a ordem é sempre: religa primeiro, libera depois.

O parâmetro saida é um ponteiro pra onde eu copio os dados do removido antes de liberar. Serve pro menu

conseguir imprimir o nome de quem saiu. Se eu não copiasse antes do free, o dado se perderia junto.

Semana 2

O que foi feito

- Implementação do TAD ListaEventos em eventos.h e eventos.c

- Implemen

Regra de impedir inscrição duplicada

Está em pa_inscriver. Antes de inserir, a função chama lp_buscar_ra na lista daquele evento. Se o RA já estiver lá, recusa e o menu mostra o erro.

Uma coisa que eu tive que pensar foi o escopo dessa regra. A verificação é por evento, não global. O mesmo RA pode estar inscrito em vários eventos diferentes, o que ele não pode é aparecer duas vezes no mesmo evento. Se eu tivesse feito a verificação global, o relatório de participação individual por RA não faria sentido nenhum, porque cada pessoa só poderia ir num evento no semestre inteiro.

Dificuldade encontrada

A parte de liberar memória me deu trabalho, porque aqui tem dois níveis. Cada evento tem a sua própria lista de participantes, e cada uma dessas listas tem os seus próprios malloc, incluindo o do sentinela. No começo eu estava liberando só os eventos e achando que estava certo.

Também errei a primeira versão do lp_destruir. Eu tinha começado o laço em cabecalho->proximo, porque na minha cabeça o sentinela "não era um nó de verdade". Só que ele é, ele levou malloc igual aos outros, e ficou vazando.

Como resolvi

Testando. Eu coloquei printf nas funções de destruição contando quantos nós estavam sendo liberados e comparei com quantos eu tinha criado. Aí ficou evidente que estava faltando um por evento, que era exatamente o sentinela.

Depois disso compilei com o AddressSanitizer pra confirmar, e aí sim deu zero vazamento. Fiz a conta no papel pra entender: um evento com 3 inscritos tem 5 blocos alocados, que são 1 do nó do evento, 1 do sentinela e 3 dos participantes. Se a conta não bate, tem vazamento.

Testes realizados

Caso testado	Esperado	Obtido
Inscriver RA já inscrito no mesmo	recusa	recusou
Inscriver o mesmo RA em outro even	aceita	aceitou
Inscriver em evento inexistente	recusa	recusou
Cadastrar evento com código repeti	recusa	recusou
Remover o primeiro inscrito da lis	remove sem erro	removeu
Remover RA não inscrito	recusa	recusou

Listar evento sem nenhum inscrito	mensagem adequada	ok
Relatório de RA sem inscrições	mensagem adequada	ok

Verificação de memória

Compilação limpa, sem nenhum aviso:

```
gcc -Wall -Wextra -pedantic -std=c11 -o sistema main.c lista.c eventos.c participantes.c
```

Verificação com AddressSanitizer e LeakSanitizer: zero vazamentos e zero erros de memória.

Segundo trecho de código explicado nas minhas palavras

```
void ev_destruir(ListaEventos *le)
{
    NoE *atual = le->inicio;
    NoE *proximo;

    while (atual != NULL) {
        proximo = atual->proximo;
        lp_destruir(&atual->inscritos);
        free(atual);
        atual = proximo;
    }

    le->inicio = NULL;
    le->tamanho = 0;
}
```

Escolhi essa porque foi onde eu errei e tive que entender direito.

A linha `proximo = atual->proximo` tem que vir antes do `free(atual)`. Se eu liberasse primeiro e só depois tentasse ler `atual->proximo`, eu estaria lendo o campo de um bloco que já voltou pro sistema, e eu perderia o caminho pro resto da lista. É o mesmo princípio da remoção: guarda o endereço que você vai precisar antes de liberar qualquer coisa.

A chamada de `lp_destruir(&atual->inscritos)` é o que eu tinha esquecido na primeira versão. Cada evento carrega a lista de participantes dele dentro da própria struct, e aquela lista tem malloc próprio. Se eu só liberasse o nó do evento, todos os participantes dele continuariam alocados sem ninguém apontando pra eles, que é o vazamento clássico.

E dentro do `lp_destruir` o laço começa no `l->cabecalho`, não no `cabecalho->proximo`. Foi o segundo erro que eu cometi. O sentinela não guarda dado útil, mas ele foi criado com malloc na inicialização, então ele precisa do `free` dele igual a qualquer outro nó.

No fim eu zero o `inicio` e o `tamanho`. Não é obrigatório porque o programa vai encerrar logo em seguida, mas deixa a struct num estado coerente e evita que alguém use um ponteiro pendurado por engano se a função for chamada no meio do programa.

Reflexão final

Antes desse trabalho eu entendia ponteiro na teoria, mas não conseguia ler uma linha com duas setas sem me perder. O que destravou foi parar de olhar o código e começar a desenhar o estado da estrutura antes e depois de cada linha, com endereços inventados. Depois que eu fiz isso umas três vezes, as operações de lista

viraram quase sempre a mesma coisa: achar o nó anterior, religar dois ponteiros na ordem certa, e só então liberar.

A escolha do nó cabeçalho se provou acertada na prática. A função de remoção ficou com um caminho só, e eu nem precisei pensar no caso de remover o primeiro inscrito, que foi justamente o caso que mais quebrou nos meus testes iniciais quando eu estava fazendo sem sentinela.

Fui honesto comigo sobre o processo: comecei mais tarde do que devia e fiz boa parte corrido perto do prazo. Isso cobrou caro na parte de memória, porque os dois erros de free que eu cometi eram erros de pressa, não de falta de entendimento. Se eu tivesse rodado o verificador de memória desde o começo em vez de só no final, teria achado os dois em cinco minutos.

Se tivesse mais tempo, eu trocava a lista de participantes por uma duplamente encadeada. O motivo é que hoje a remoção precisa percorrer a lista pra achar o nó anterior, e com o ponteiro ant em cada nó eu chegaria no anterior num salto só. Isso levaria a remoção de $O(n)$ para $O(1)$ nos casos em que eu já tenho o ponteiro do nó, e ainda permitiria percorrer a lista de presença de trás pra frente, que seria útil pra conferir chamada. O custo seria um ponteiro a mais por participante e mais atenção pra manter os dois sentidos sempre consistentes.